

Intelligent Data Processor Core Foundation

Complete Guide: Document Summary, Q&A, and Beginner Project Walkthrough

(Large-Text Easy-to-Read Edition)

This document combines everything covered so far: a full question-and-answer review of the original proposal, a complete beginner's guide to actually executing this project (purpose, requirements, database choice, topics to learn, and step-by-step instructions), and the full working demo code.

Part 1: Source Document — Full Q&A Review

Section I – Processing Engine Overview

This section explains how to design an AI engine that builds and manages ETL data pipelines for partner data, instead of relying on hand-coded scripts.

1. What are the three tiers of an AI-driven data processing engine's architecture?

The architecture has three tiers: (1) the Orchestration Layer, which schedules and manages task execution; (2) the Agentic / LLM Layer, the "engine" itself, which is an AI agent connected to data sources that can generate SQL or Python code; and (3) the Execution & Storage Layer, where data is actually processed and stored.

2. What role does the Orchestration Layer play, and which tools are given as examples?

The Orchestration Layer manages the execution of tasks using traditional schedulers or workflow managers. The examples given are Apache Airflow and Kestra (on Shakudo).

3. What is the function of the "Agentic / LLM Layer," and what does it use to generate SQL or Python code dynamically?

This layer is the core "engine" of the system. It consists of an AI agent connected to data sources and equipped with APIs that let it dynamically generate SQL or Python code, rather than relying on pre-written scripts.

4. Which platforms are mentioned as part of the Execution & Storage Layer?

Databricks, Snowflake, and PySpark are mentioned as the destinations and compute engines where data is physically moved and processed.

5. What are Dynamic Connection Tools used for in this system?

They allow the AI agent to list available databases, fetch table schemas, and safely authenticate API connections, essentially giving the agent the ability to explore and read data specifications on its own.

6. What example is given for a system prompt used to instruct the AI agent?

The example system prompt is: "You are a data engineering assistant. Given the source schema X and destination schema Y, write the SQL or PySpark code to clean the data and load it into the warehouse."

7. What is the purpose of Enrichment Functions, and what two AI functions are mentioned as examples?

Enrichment Functions use AI to parse messy, unstructured data (such as support tickets or reviews) before loading a structured summary into the database. The two example functions mentioned are `ai_classify` and `ai_extract`.

8. List the four steps in the step-by-step implementation of a basic AI ETL agent.

Step 1: Define the Source & Destination — set up API/JSON sources and database connections (e.g., PostgreSQL or Snowflake). Step 2: Initialize the Agent Tools — write Python functions that let the LLM read schemas and execute Python/SQL code. Step 3: Run the AI Agent — pass a user prompt to the LLM so it can request the schema, map columns, and write transformation logic. Step 4: Add Validation — integrate a schema validator or data quality checker before committing the transaction.

9. Why is a schema validator or data quality checker important in Step 4 (Add Validation)?

Because the AI generates its own transformation logic, a validation step is needed to make sure the LLM's output conforms to the database's constraints before the data is actually committed, preventing errors or corrupted data from reaching the destination.

10. Name three enterprise platforms mentioned that offer built-in AI-augmented ETL capabilities, and briefly describe what each contributes.

Databricks offers Lakeflow and Agentic AI functions (such as `ai_extract` and `ai_classify`) that embed AI models directly into ETL workflows. Matillion and Airbyte provide AI-assisted schema mapping and SQL optimization, reducing the manual effort of building pipelines. Informatica offers AI-powered mapping recommendations along with a conversational natural-language interface for generating pipelines.

11. What two clarifying questions does the document suggest asking before building a custom engine from scratch?

(1) What frameworks do we prefer to use, such as LangChain or LlamaIndex? (2) What are the primary data sources, such as APIs, local files, or relational databases?

Section II – Processing Engine Overview

This section covers the underlying tools and frameworks used to build or run the AI models themselves, either from scratch or by hosting existing open-source models locally.

12. What two general approaches are suggested for building a custom AI engine (in terms of frameworks)?

The document suggests either using a foundational machine learning framework (like PyTorch or TensorFlow) to build and train models, or using a self-hosted inference engine (like Ollama or vLLM) to run existing large language models locally on your own hardware.

13. What is the main benefit of using a self-hosted inference engine like Ollama or vLLM?

Running LLMs locally on your own hardware ensures data privacy and gives you total customization over the model and how it is used, rather than depending on a third-party API.

14. What distinguishes Ollama from vLLM in terms of use case?

Ollama is designed to be the easiest way to run and serve open-source LLMs locally (such as Llama 3 or Mistral) by bundling model weights and configuration into a single package. vLLM, on the other hand, is a high-throughput, memory-efficient serving engine optimized for production deployments that scale well on GPUs.

15. What makes LocalAI unique compared to Ollama and vLLM?

LocalAI is a self-hosted, drop-in replacement for the OpenAI API. It lets you run local LLMs, generate images, and transcribe audio on consumer-grade hardware, notably without requiring a GPU.

16. How do PyTorch and TensorFlow differ in their typical use cases?

PyTorch is a Python-based deep learning framework widely used for AI research, rapid prototyping, and dynamic model training. TensorFlow is an end-to-end platform developed by Google, better suited for numerical computation and deploying machine learning models across web, mobile, and edge devices.

17. What is Keras, and what frameworks can it run on top of?

Keras is a high-level neural networks API written in Python that is designed to be approachable for beginners. It can run on top of TensorFlow, PyTorch, or JAX.

Part 2: Beginner's Project Guide

2.1 Purpose of the Project

The purpose is to build a system where an AI agent — not a human engineer — handles the work of moving and cleaning data (ETL). Instead of writing fixed scripts, you give an LLM tools (functions it can call) and a goal, and it reads the data structure, figures out the transformation logic, writes the code, and loads the data itself.

2.2 Project Goal, Output, and Outcome

Goal	Build a working AI agent that migrates and cleans data between two database tables without manually writing transformation scripts.
Output	A runnable Python project: a demo database plus an agent script that performs the migration.
Outcome	Hands-on understanding of agentic AI, tool-calling, and AI-driven ETL — enough to confidently discuss, extend, or explain the concept.

2.3 Requirements

- Python 3.10 or newer
- pip (Python's package manager)
- An LLM API key — Anthropic (Claude) or OpenAI — free tiers are available
- Python packages: langchain, langchain-anthropic (or langchain-openai)
- A database — see 2.4 below
- A code editor (VS Code is recommended for beginners)

2.4 Which Database Should You Use?

For learning: use SQLite. It is a single file, built directly into Python, and needs no server or installation — the fastest way to practice the concept.

For real company use later: PostgreSQL, Snowflake, or Databricks, as mentioned in the original document. Do not start there — learn the concept on SQLite first, then move to a real database once you understand how the agent works.

2.5 Topics You Should Learn (in order)

- Python basics — functions, variables, imports (you likely already have this)
- Basic SQL — SELECT, INSERT, UPDATE, CREATE TABLE
- What an LLM "agent" is — a model that can call functions/tools instead of only replying with text; this is the single most important concept in this whole project
- Tool-calling / function-calling — how you expose Python functions to an LLM so it can use them
- LangChain fundamentals — the framework used to build the agent (models, tools, agents)
- Prompt engineering basics — how to write a system prompt that gives the AI a clear job
- ETL concepts — Extract, Transform, Load, and why data cleaning matters
- Data validation — why you must check the AI's output before trusting it
- (Later) Local LLM hosting — Ollama or vLLM, only needed if data privacy requires running models on your own hardware instead of an API

2.6 Does This Project Require Coding Skill?

Yes, but only basic-to-intermediate Python. You do not need machine learning or AI research experience. The LLM does the "hard" reasoning work — your job is to build the scaffolding (tools, prompt, validation) around it.

2.7 Step-by-Step: How To Actually Do This Project

- Step 1 — Install packages: `pip install langchain langchain-anthropic`
- Step 2 — Get an API key from console.anthropic.com (or platform.openai.com for OpenAI) and set it as an environment variable
- Step 3 — Create a demo database with a messy source table and a clean destination table (the `setup_demo.py` script below does this for you)
- Step 4 — Build agent tools — plain Python functions: `list_tables`, `get_schema`, `preview_data`, `run_sql`
- Step 5 — Write a system prompt telling the AI its job: read the schemas, clean the data, migrate it
- Step 6 — Run the agent and watch it inspect the schema, write SQL, and execute it
- Step 7 — Validate the result — check row counts and required fields before trusting the output
- Step 8 — Once comfortable, swap SQLite for a real database (PostgreSQL/Snowflake) and add more realistic messy data

Part 3: Full Working Demo Code

The following two scripts are a complete, tested, runnable demo of this entire project. `setup_demo.py` was tested and confirmed working. Copy each into its own .py file with the filename shown.

File: setup_demo.py

Creates a demo SQLite database with a messy source table and an empty clean destination table.

```
"""
setup_demo.py
-----
Creates a small demo SQLite database with:
- a messy "source_customers" table (like real partner data: inconsistent
  casing, extra whitespace, mixed date formats)
- an empty, clean "destination_customers" table with a defined schema

Run this once before running ai_etl_agent.py
"""
import sqlite3

DB_PATH = "demo_partner_data.db"

def setup_database():
    conn = sqlite3.connect(DB_PATH)
    cur = conn.cursor()

    # Drop tables if they already exist (so you can re-run this safely)
    cur.execute("DROP TABLE IF EXISTS source_customers")
    cur.execute("DROP TABLE IF EXISTS destination_customers")

    # ---- Messy SOURCE table (this simulates real "partner data") ----
    cur.execute("""
        CREATE TABLE source_customers (
            id INTEGER PRIMARY KEY,
            full_name TEXT,
            EMAIL TEXT,
            signup_date TEXT,
            country TEXT
        )
    """)

    messy_rows = [
        (1, " john smith ", "JOHN.SMITH@EXAMPLE.COM", "2023-01-15", "usa"),
        (2, "Maria Garcia", "maria.g@example.com", "15/02/2023", "Spain"),
        (3, " AHMED KHAN", "ahmed.khan@example.com ", "2023-03-10", "pakistan"),
        (4, "Li Wei", "LI.WEI@EXAMPLE.COM", "2023/04/05", "china"),
        (5, "sara ahmed", "sara.ahmed@example.com", "10-05-2023", "PAKISTAN"),
```

```

]
cur.executemany(
    "INSERT INTO source_customers VALUES (?, ?, ?, ?, ?)", messy_rows
)

# ---- Clean, well-structured DESTINATION table ----
# Notice the schema is DIFFERENT from the source on purpose:
# different column names, capitalized country codes, etc.
cur.execute("""
    CREATE TABLE destination_customers (
        customer_id INTEGER PRIMARY KEY,
        name TEXT,
        email TEXT,
        signup_date TEXT,
        country_code TEXT
    )
""")

conn.commit()
conn.close()
print(f"Demo database created: {DB_PATH}")
print("Tables: source_customers (messy, 5 rows), destination_customers (empty, clean)")

if __name__ == "__main__":
    setup_database()

```

File: ai_etl_agent.py

The AI agent itself: tools, system prompt, agent execution, and validation.

```

"""
ai_etl_agent.py
-----
A minimal working example of the "AI-driven ETL agent" described in the
Intelligent Data Processor Core Foundation document.

WHAT THIS DOES (matches the document's 4-step implementation):
  Step 1: Source & destination already exist (demo_partner_data.db)
  Step 2: We give the agent TOOLS: list_tables, get_schema, run_sql
  Step 3: We give the agent a PROMPT and let it figure out the mapping
          and write the transformation SQL itself
  Step 4: We VALIDATE row counts before/after so nothing silently breaks

BEFORE RUNNING:
  1. Run setup_demo.py first to create the demo database.
  2. Install dependencies:
      pip install langchain langchain-anthropic
  3. Set your API key as an environment variable:
      export ANTHROPIC_API_KEY="your-key-here"    (Mac/Linux)
      set ANTHROPIC_API_KEY=your-key-here         (Windows)

RUN:
  python ai_etl_agent.py
"""

```

```

import os
import sqlite3
from langchain.agents import create_agent
from langchain_anthropic import ChatAnthropic

DB_PATH = "demo_partner_data.db"

# -----
# STEP 2: AGENT TOOLS
# These are plain Python functions. The LLM decides WHEN to call them.
# This is the core idea of "agentic AI" -- the model can take actions,
# not just generate text.
# -----

def list_tables() -> str:
    """List all tables in the database."""
    conn = sqlite3.connect(DB_PATH)
    cur = conn.cursor()
    cur.execute("SELECT name FROM sqlite_master WHERE type='table'")
    tables = [row[0] for row in cur.fetchall()]
    conn.close()
    return ", ".join(tables)

def get_schema(table_name: str) -> str:
    """Get the column names and types for a given table."""
    conn = sqlite3.connect(DB_PATH)
    cur = conn.cursor()
    cur.execute(f"PRAGMA table_info({table_name})")
    columns = cur.fetchall()
    conn.close()
    if not columns:
        return f"Table '{table_name}' not found."
    return "\n".join([f"{col[1]} ({col[2]})" for col in columns])

def preview_data(table_name: str, limit: int = 5) -> str:
    """Preview a few rows of data from a table so the agent can see real values."""
    conn = sqlite3.connect(DB_PATH)
    cur = conn.cursor()
    cur.execute(f"SELECT * FROM {table_name} LIMIT {limit}")
    rows = cur.fetchall()
    col_names = [description[0] for description in cur.description]
    conn.close()
    result = f"Columns: {col_names}\n"
    for row in rows:
        result += f"{row}\n"
    return result

def run_sql(query: str) -> str:
    """Execute a SQL statement (INSERT, UPDATE, SELECT, etc.) against the database."""
    conn = sqlite3.connect(DB_PATH)
    cur = conn.cursor()
    try:
        cur.execute(query)
        conn.commit()
        if query.strip().upper().startswith("SELECT"):

```

```

        rows = cur.fetchall()
        return str(rows)
    return f"Success. Rows affected: {cur.rowcount}"
except Exception as e:
    return f"SQL ERROR: {e}"
finally:
    conn.close()

# -----
# STEP 4: VALIDATION (run this AFTER the agent finishes, before you trust it)
# -----

def validate_migration():
    """Simple data-quality check: does the destination row count make sense,
    and are there any empty required fields?"""
    conn = sqlite3.connect(DB_PATH)
    cur = conn.cursor()

    cur.execute("SELECT COUNT(*) FROM source_customers")
    source_count = cur.fetchone()[0]

    cur.execute("SELECT COUNT(*) FROM destination_customers")
    dest_count = cur.fetchone()[0]

    cur.execute("SELECT COUNT(*) FROM destination_customers WHERE email IS NULL OR email = ''")
    missing_emails = cur.fetchone()[0]

    conn.close()

    print("\n--- VALIDATION REPORT ---")
    print(f"Source rows: {source_count}")
    print(f"Destination rows: {dest_count}")
    print(f"Missing emails: {missing_emails}")

    if dest_count != source_count:
        print("WARNING: row counts do not match. Do not trust this migration blindly.")
    elif missing_emails > 0:
        print("WARNING: some destination rows are missing emails.")
    else:
        print("Validation passed: row counts match and required fields are populated.")

# -----
# STEP 3: RUN THE AGENT
# -----

def main():
    if not os.environ.get("ANTHROPIC_API_KEY"):
        print("ERROR: Please set your ANTHROPIC_API_KEY environment variable first.")
        return

    llm = ChatAnthropic(model="claude-sonnet-4-6", temperature=0)
    tools = [list_tables, get_schema, preview_data, run_sql]

    system_prompt = (
        "You are a data engineering assistant. You have access to a SQLite database "

```

```

        "with a messy 'source_customers' table and a clean, empty 'destination_customers'
        table with a different schema. "
        "Your job: inspect both schemas, preview the source data, then write and run "
        "SQL to clean the data (fix casing, trim whitespace, standardize date format to "
        "\"YYYY-MM-DD, uppercase country codes) and load it into destination_customers. "
        "Map columns sensibly even though names differ between source and destination. "
        "Use your tools step by step: list_tables, get_schema, preview_data, then run_sql
        to perform the migration."
    )

    agent = create_agent(llm, tools, system_prompt=system_prompt)

    user_prompt = "Please migrate and clean the data from source_customers into destination_customers"
    print("Running AI ETL agent...\n")
    result = agent.invoke({"messages": [{"role": "user", "content": user_prompt}]})

    # Print the agent's final response
    final_message = result["messages"][-1]
    print("\n--- AGENT FINAL RESPONSE ---")
    print(final_message.content)

    # Step 4: validate what it actually did
    validate_migration()

if __name__ == "__main__":
    main()

```

2.8 How to Run This Code

1. Save both files in the same folder
2. In a terminal, run: `pip install langchain langchain-anthropic`
3. Set your API key: `export ANTHROPIC_API_KEY="your-key-here"` (Mac/Linux) or set `ANTHROPIC_API_KEY=your-key-here` (Windows)
4. Run: `python setup_demo.py` (creates the database — no cost, works offline)
5. Run: `python ai_etl_agent.py` (this calls the LLM — small API cost, a few cents)

Note: running `ai_etl_agent.py` requires a valid API key and will use a small amount of API credit. `setup_demo.py` is free and works without any API key.